Gpap — **G**PU-**P**arallel **A**ssumption **P**ropagation

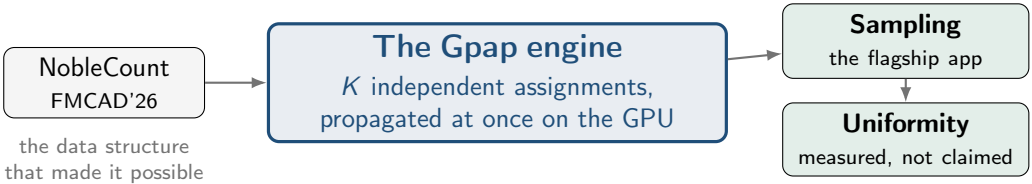
A K-parallel BCP engine — and guaranteed-valid SAT solution sampling built on it

Yogev Shalmon

Technion — Faculty of Data and Decision Sciences

Project status — August 8, 2026

The headline: a BCP engine that runs 300× a CPU core



The result

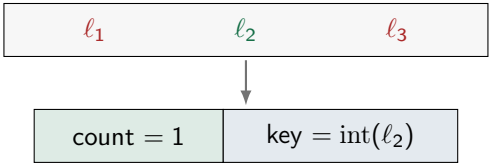
$\sim 10^9$ **propagations/s** vs. $\sim 5 \times 10^6$ on a single CPU core

197–631× — and **essentially flat across every benchmark family**

But an engine alone is not a contribution. So we built the application that needs exactly this shape of parallelism — **SAT solution sampling** — and measured it against the state of the art.

Where this starts: NobleCount (FMCAD'26)

a clause, one literal currently true



two fields per clause, updated branch-free on every flip

count $= 1 \Rightarrow$ the key **is** the surviving literal
$\Rightarrow$ **no clause scan, ever**

Dropped into ProbCCAnr, the SOTA local-search solver. **No heuristic changes, no re-tuning.**

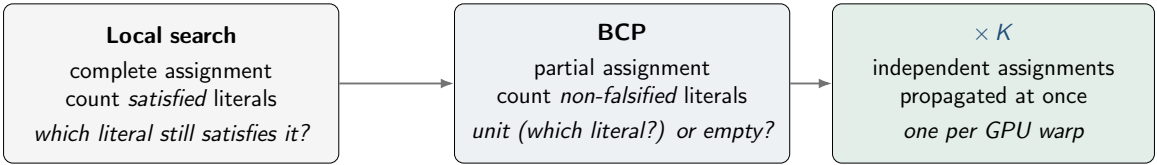
New state of the art on **FCC** (PAR2 -24.8%) *and* **SC22**.

A data-structure win, orthogonal to heuristic design.

Its own closing sentence

"... making it **a promising candidate for parallel clause-state maintenance in SIMD- or GPU-based implementations**."

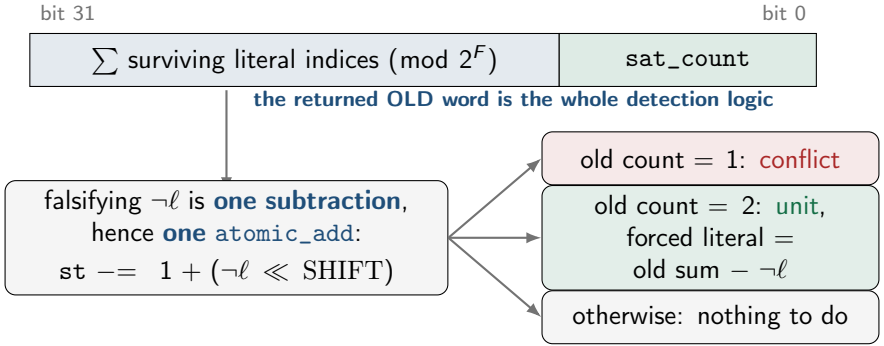
Same idea, different question



The primitive, in one line

Negated occurrence lists: setting ℓ true falsifies $\neg\ell$ in *exactly* the clauses of $\mathrm{occ}[\ell]$ — a decision touches only what it can affect.

One word per clause — and the update is its own detector



The **assignment** is the same trick: a bitmap where ℓ and $\neg\ell$ share a word, so one `atomic_or` returns duplicate *and* contradiction together.

On a CPU, XOR and sum are equivalent — NobleCount used XOR. On a GPU they are not: the sum makes the update an atomic whose *return value* resolves every race — no locks, no `shfl`/`ballot`.

A one-slide GPU primer: SMs, warps, and why occupancy is everything

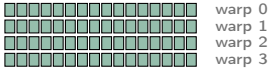
A GPU is many cores (SMs)



L4: **58** · A100: **108**

Each SM juggles many warps

a warp = **32 threads in lockstep**



Memory is ∼400 **cycles away.** A GPU does not hide that with caches — it hides it by having **another warp ready to run** the moment one stalls.

⇒ **You must supply enough independent work.**

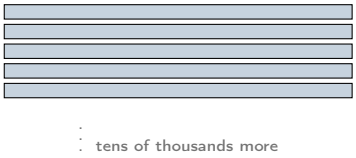
Too few warps and the machine sits idle waiting on DRAM, no matter how fast the arithmetic is.

"Occupancy" = resident warps / the maximum the SMs can hold.

This is the whole design constraint for what follows: **our unit of parallelism is one assignment** — so the question is always *how many can we keep in flight?*

The architecture: one assignment per warp

Small formula ⇒ K **huge** (∼100k rows)

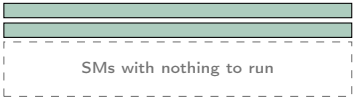


GPU oversubscribed — a stall is hidden by the next row. More compute per row buys nothing: we are **waiting on memory**.



SCALAR: **1 lane of 32** walks $\mathrm{occ}[\ell]$ — no atomics needed

Big formula ⇒ VRAM caps K **low** (hundreds)



GPU starved — too few rows to hide latency, most of the machine idles. **Now we *want* more work per row.**

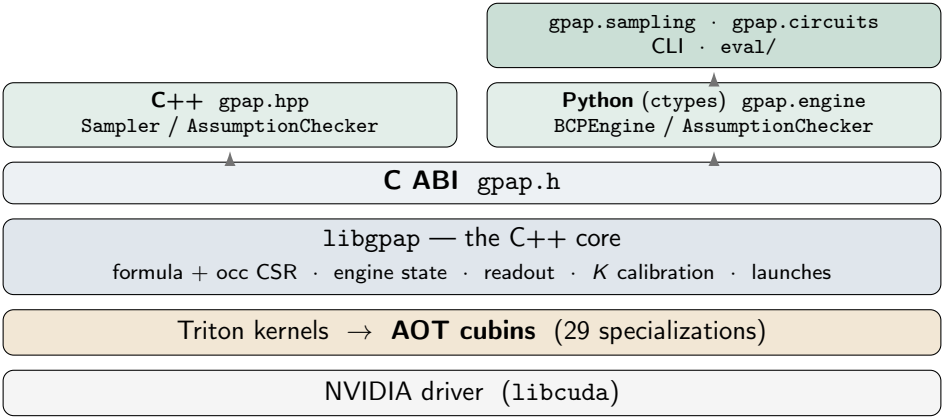


WAVE: **all 32 lanes** update clauses *inside one assignment*

One dispatch rule, measured: WAVE while $K < 30\times$ resident warps, SCALAR beyond.

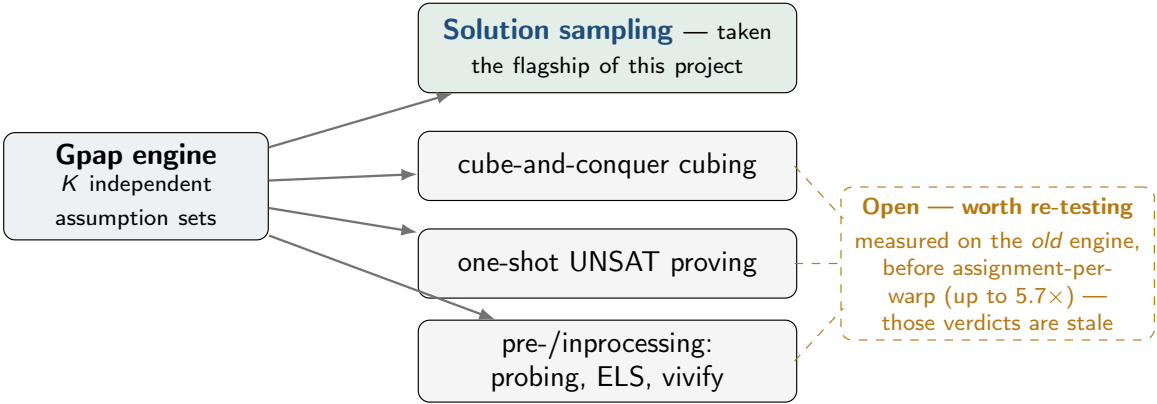
Same state, bit-identical results — the formula size that starves one variant is exactly what feeds the other.

The tool



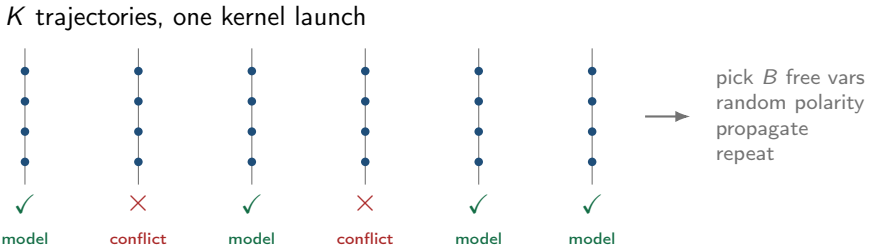
One implementation: both faces link the same library, so there is no second engine to drift. The build needs no CUDA toolkit and no `nvcc`; the runtime needs `libcuda` alone.

What is a K-parallel BCP primitive good for?



Sampling fits the layout exactly: millions of independent randomized completions over one shared, read-only occurrence structure.

The sampler: inspired by CMSGen, run K at a time



Inspired by CMSGen's Algorithm 1 — random variable, random polarity, propagate — but K trajectories at a time.

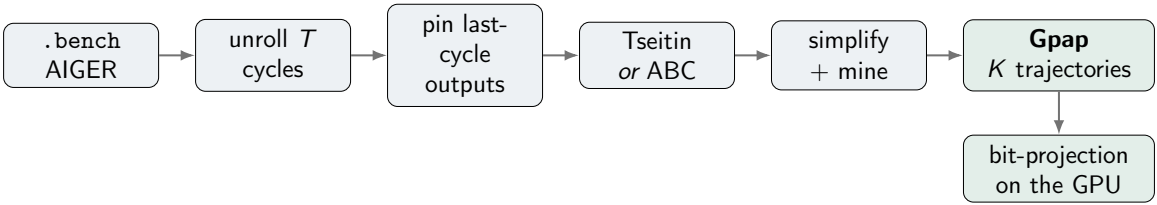
The structural difference: CMSGen backtracks out of a conflict; Gpap *discards* the trajectory.

Two consequences

Every emitted row is a real model — no gradient, no rounding, no repair.

Discarding *is* rejection sampling ⇒ an exactness knob (later).

The flagship pipeline

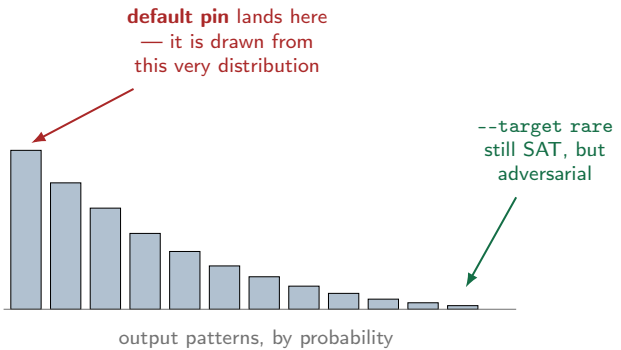


Support = primary inputs × cycles. Only the bit matrix leaves the device.

Encoder is a choice: native Tseitin, or ABC. Same projected solution set — gated by set-equality tests.

Config is per instance: a bandit measures at full K. The right answer is instance-dependent, so we never guess it.

The question we asked before a reviewer could: why not just simulate?



At default pins, plain CPU simulation + filtering beat us on 17/19.

So we changed the protocol, not the claim: pin a *rare* output pattern — still reachable, but adversarial for rejection.

At rare targets it cannot play at all: on EPFL, 0 hits in ∼10M random patterns while Gpap samples at 89–100% success.

Rejection is not just a rival — it picks our targets and audits every emitted sample.

Headline: Gpap vs. CMSGen, 357 cells, 0 errors

family	targets	**Colab L4**	4080 laptop	wins
ITC'99 + ISCAS-89 seq, T=25	**rare**	39.1×	36.5×	17/18
ISCAS-85 combinational	**rare**	615×	538×	10/10
HWMCC'17, T=25	**rare**	196×	156×	7/7
EPFL combinational	**rare**	9.4×	6.4×	13/17
Sketch / `sk_` CNFs	—	98.9×	46.2×	27/29

Every ratio is higher on the L4 — and none of that is Gpap.

Paired on identical cells: Gpap 0.96×, **CMSGen** 0.41×. The Colab host is a slower CPU, and CMSGen is single-core.

The rule we adopted: never quote a ratio without the CPU.

Otherwise a reviewer reads 98.9× and correctly calls it a rigged baseline.

357 cells, 0 skipped, 0 errors, 0 unsound. Per instance at its best encoder; geomeans exclude tautological pins and instances that need no search. Full tables in the appendix.

Is the speedup the engine, or the benchmark?

	implications/s		implications/sample	
family (**rare**)	Gpap	CMSGen	Gpap	CMSGen
ISCAS-85 combinational	1.05×10^9	1.67×10^6	517	507
ITC'99/ISCAS-89, T=25	1.14×10^9	5.20×10^6	30,538	5,422
HWMCC'17, T=25	1.05×10^9	4.58×10^6	12,615	10,737
EPFL combinational	1.08×10^9	2.14×10^6	12,139	226
Sketch / `sk_` CNFs	1.00×10^9	4.23×10^6	851	355

The engine's rate is flat — $\sim10^9$/s everywhere, spread only 1.41×, while the *sample* ratio moves ∼2 orders.

It is a property of the GPU, not the benchmark.

$$\text{samples/s ratio} = \frac{\text{impl/s ratio}}{\text{impl/sample ratio}}$$

Exact on 9/9 families. The engine buys 197–631×; rejection spends part back — 1.0× on ISCAS-85, 5.6× on sequential, 53.7× on EPFL.

We discard a conflicted trajectory; CMSGen backtracks and keeps the partial work. EPFL pays the most — and is the one family with an instance we could not sample at all.

Who else puts SAT sampling on a GPU?

DEMOTIC
ASP-DAC'25

ISCAS-85, their own output pins

Recurrent-CircuitSAT
MLCAD'25

closest to our domain: sequential circuits

HTS
arXiv 2502.08673

CNF, hybrid tree search

They all do the same thing: relax the circuit to a *continuous* function and follow the gradient towards a solution.

Fast, and GPU-native — but a gradient lands *near* a solution, so the output must be rounded and repaired, and may not satisfy the formula.

Gpap is discrete throughout: every trajectory is real propagation, so every emitted row is a model by construction — nothing to verify, nothing to repair.

We compare on *their* metric: unique solutions per second of their own reported runtime.

Versus the GPU gradient samplers

rival	pins used	GPAP / rival	GPAP wins
DEMOTIC	*their* ISCAS-85 pins	2.5×	19/20
DEMOTIC	our **rare** pins	2,427×	16/16
Recurrent	*their* target table	4.4×	6/6
Recurrent	our **rare** pins	635×	5/6
HTS	our CNFs	1,584×	13/14

The protocol matters more than the rival.

Recurrent-CircuitSAT reads 4.4× on *their* pins and 635× on ours — a 140-fold swing, same sampler, same GPU, same day. Only the target changed.

Their pins are easy by construction, so a win there is not a sampling result.

Honest losses: the "service" family — jointly-constrained instances that need clause *learning*.

7 rival runs failed (timeout / no gradient path): reported, never counted as zeros. Cells where *we* emitted nothing are counted as losses.

Does that verdict survive a different GPU?

rival	protocol	GPAP / rival (geomean)		GPAP wins
		4080 laptop	**L4**	on L4
DEMOTIC	their ISCAS-85 pins	2.9×	2.5×	19/20
DEMOTIC	our **rare** pins	7,704×	2,427×	16/16
Recurrent	our **rare** pins	599×	422×	3/3
HTS	our CNFs, both encoders	—	2,042×	10/10

What transfers: the verdict.

Win counts are *identical* on both cards, instance by instance — because both sides share the GPU.

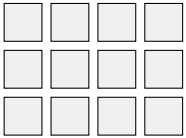
What does not: the magnitude.

At rare pins the rival scores ≈0, so the ratio has a near-zero denominator. **Read these as orders of magnitude, never as constants.**

Honest: c1355 loses on both cards; at a rare b11 pin *neither* sampler finds anything.

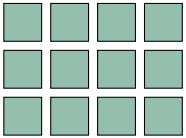
What "uniform" means — and why a sampler lives or dies by it

all solutions of the formula



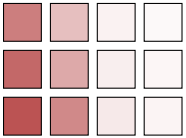
each square = one model

uniform



every model equally likely

skewed



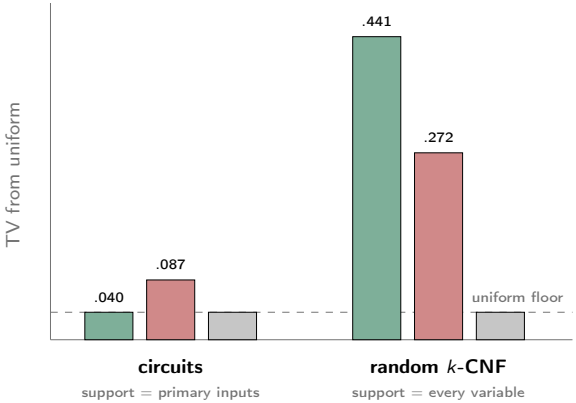
a fast sampler that keeps returning the same corner

Why it matters: sampling is used to *estimate* things — coverage, failure probability, test stimuli. A skewed sampler answers those questions wrongly, no matter how fast it is.

Soundness $\neq$ uniformity. Every row we emit *is* a model — that is free for us. Being drawn with the *right probability* is a separate claim, and it has to be measured.

Standard tool: Barbarik. Ours: an exact oracle (next slide).

Uniformity: measured against an exact oracle



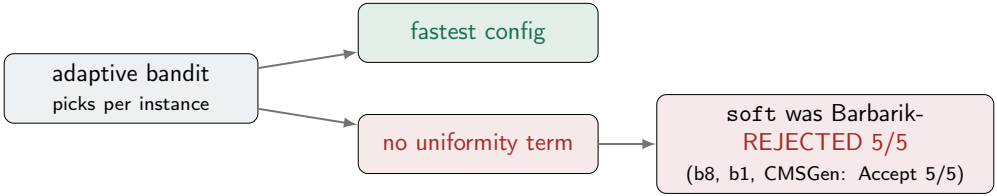
green = Gpap, red = CMSGen, grey = exact-uniform control.

The instrument: generate small formulas, enumerate their solutions exhaustively (solver-free), test χ^2 + TV against the *known* uniform. Calibrated: 0/120 false positives on an exact-uniform control, 97% power at TV 0.05. 600 instances, 10,800 runs, zero unsound rows.

On the flagship regime we are *at* the floor — and CMSGen is not. On random k-CNF we are worse. **Never quote the aggregate.**

An exactness knob: at $B \geq |\text{support}|$ one round fixes the whole support and we discard failures — exact rejection sampling, 24/24 at TV 0.0392. Bounded by yield when p_{accept} is small.

The uncomfortable one: our own bandit optimizes the wrong thing



Acted on: `soft` is opt-in behind `--allow-soft`.

The price, on the published comparison — the whole 357-cell matrix run *twice*, one flag changed: up to 4.6× on deep sequential unrolls (14.0→64.9× at T=50), ≈1.0× on ISCAS-85, `sk_` and HWMCC-random.

safe-arm success	cells	safe/soft	`soft` chosen
$< 10^{-4}$	13	0.119×	5/13
10^{-4}–10^{-2}	22	0.313×	12/22
10^{-2}–0.5	86	0.822×	24/86
> 0.5	196	0.988×	25/196

Nearly free where the sampler already works; expensive only where it does not — exactly where `soft` wins.

The single geomean (0.796×, headline 1.30×) pools regimes and is the least useful number. Control: CMSGen 1.004× between arms, 0/357 cells differ in M or |support|.

Status, and what is open

Done

- engine: packed kernels, scalar + WAVE, `libgpap`, three language faces
- sampler: pipeline, portfolios, mining, ELS, GPU projection
- front-ends: `.bench`, AIGER, ABC, simulation
- evaluation: resumable sweep, 3-seed medians, fair CMSGen, 3 GPU rivals
- uniformity: 3-tier readout, Barbarik, exact-oracle fuzzing

Open — build

- **uniformity-aware config choice** — the one item that changes what a user gets
- per-instance encoder auto-pick
- state arena ($\sim$1.2$\times$)
- ROCm port (no kernel work)
- re-test the other applications

Open — measure

- **A100 rerun** — every number is K-bound on a 12 GB card
- multi-core CMSGen, QuickSampler
- break-even simulation throughput
- Barbarik $\times$10 / ScalBarbarik
- one single-session ablation

Turning this into a paper

A GPU-native clause-state primitive	one atomic's return value *is* the unit/conflict detection
A batched-assumption BCP engine	K independent assignments, one per warp + a measured dispatch rule
A sampler	guaranteed-valid; 12–538× over CMSGen, 10^2–10^5× over GPU rivals at rare targets
A measurement standard	rejection baseline, target rarity, vacuous pins, emission fairness, uniformity

Have: every figure on these slides, the 357-row sweep, three rival tables, the uniformity campaign.
Need: the A100 column, a multi-core CMSGen row, one single-session ablation, and a decision on the bandit/uniformity trade.

Backup: the engine, in features

- one packed `int32` per clause — update is a single atomic
- assignment as a bitmap — one atomic gives dedup *and* contradiction
- two propagate kernels, **auto-dispatched** per formula
- the whole decide+propagate trajectory **fused into one launch**
- K sized analytically from free VRAM; unit-flush cached at construction
- readout stays **on device** — only counts and projections cross the bus
- two packed splits chosen per formula; out-of-envelope formulas rejected, never silently degraded

Bottom line

Up to 5.7× end-to-end over the pre-WAVE engine — with **bit-identical model sets** at every step.

Negative results, kept because they cost weeks: classic **2-watched-literals on GPU is 1.8–2.6× slower** than NobleCount-style counting, on ∼5× the per-row state; 1-byte clause state, clause renumbering and multi-warp tiles all measured out.

Backup: the same suite on the 4080 laptop

family	targets	GPAP / CMSGen	wins
ITC'99 + ISCAS-89 sequential, T=25	**rare**	36.5×	17/18
ITC'99 + ISCAS-89 sequential, T=25	random	105.8×	18/18
ISCAS-85 combinational	**rare**	538×	10/10
HWMCC'17 (curated), T=25	**rare**	156×	7/7
EPFL combinational	**rare**	6.4×	13/17
Sketch / `sk_` projected CNFs	—	46.2×	27/29

RTX 4080 laptop (`i7-13800H`) vs. single-core patched CMSGen; per instance at its best encoder. Superseded by the L4 sweep in the main deck — kept because it is the machine the engine was developed on.

Corrected 2026-08-04/05, by one rule: **a cell that measures the instance instead of the sampler does not belong in a geomean**. (i) vacuous output pins (`b02`/`b05`/`b07`/`s526` are constant at T=25, so pinning them is a tautology): including them read 47.7×, an inflation of up to 1.9× falling almost entirely on the rare column. (ii) instances that sample with **zero decisions and zero implications** — the support is free after propagation, so no search happens, and the best-encoder rule *prefers* them because they carry the highest ratio: EPFL read 12.7×, `sk_` 48.5×.

Backup: numbers on hand

Engine

- vs pre-WAVE: b14 5.7×, b12 2.8×, b04 1.8×, b03 1.5×
- rates: b03 631k, b06 945k, b04 81k, b12 50k, b14 4.0k attempts/s
- C++ port: 1.013× geomean, identical model sets
- portability: 75/1273 CUDA lines, 18 driver calls, 29/29 specializations cross-compile

Encoder

- ABC shrink 2.2–7.8× clauses; at rare targets bimodal (unlocks b14, collapses b13)
- `fast` vs `full`: b15 encode 99 s → ≤4 s

Uniformity

- 600 instances, 10,800 runs, 0 unsound; controls 0/600 FP, 600/600 power
- $B \geq |\mathrm{sup}|$: 24/24 ok, TV 0.0392 (theory floor 0.0399)
- circuits 0.040 vs CMSGen 0.087; CNF 0.441 vs 0.272
- `soft`: Reject 5/5 in 156–936 samples, TV median 0.694

Baselines / method

- CMSGen emission = 5.3× its wall (373 MB on 27.sk)
- rare-target amplification 500–500,000×
- SM clock 615–1710 MHz while >90% busy; draw noise ±40%

Appendix: every CMSGen family measured on the L4

family	targets	inst	**GPAP/CMSGen**	wins	C=0	G=0	both
ITC'99 + ISCAS-89 seq, T=25	**rare**	19	39.1×	17/18	1	0	1
ITC'99 + ISCAS-89 seq, T=50	**rare**	19	14.0×	15/16	3	0	3
ITC'99 + ISCAS-89 seq, T=25	random	19	183.3×	18/18	1	0	1
ITC'99 + ISCAS-89 seq, T=50	random	19	147.0×	19/19	7	0	0
ISCAS-85 combinational	**rare**	10	615.4×	10/10	1	0	0
ISCAS-85 combinational	random	10	636.1×	10/10	1	0	0
HWMCC'17, T=25	**rare**	7	195.6×‡	7/7	5	0	0
HWMCC'17, T=50	**rare**	7	421.5×‡	7/7	6	0	0
HWMCC'17, T=25	random	7	263.8×‡	7/7	5	0	0
HWMCC'17, T=50	random	7	520.6×‡	7/7	6	0	0
EPFL combinational	**rare**	20	9.4×	13/17	1	1	3
EPFL combinational	random	20	8.2×	12/17	1	1	3
Sketch / `sk_` projected CNFs	—	29	98.9×	27/29	0	1	0

357 cells, 193 instance-slots, 0 skipped, 0 errors. GPU **NVIDIA L4**, CPU **Intel(R) Xeon(R) CPU @ 2.20GHz**. Geomeans already EXCLUDE b02/b05/b07/s526 (outputs constant at T=25, so pinning is a tautology) and any instance that samples with zero search. **The zero columns are symmetric**: **C=0** instances CMSGen could not sample (our wins), G=0 instances *we* could not (our losses), **both** neither could. None has a finite ratio, so none enters a geomean — and the four counts plus the geomean's instances sum to **inst**. ‡geomean over <3 instances (CMSGen scored zero on the rest): an illustration, not an estimate.

Appendix: every GPU rival measured on the L4

rival	protocol	cells	GPAP/rival	wins	R=0	G=0	failed
DEMOTIC	their ISCAS-85 pins	20	2.5×	19/20	0	0	0
DEMOTIC	our **rare** pins	18	2,426.6×	16/16	0	0	0
Recurrent	their target table	9	4.4×	6/6	0	0	3
Recurrent	our **rare** pins	9	634.9×	5/6	1	0	3
HTS	our CNFs (both encoders)	18	1,584.4×	13/14	2	1	3

Their metric: unique solutions / their `model_runtime`, which excludes their model build while ours includes construction. **failed** = timeout or no gradient path; those rows are reported, never folded in as a zero. **R=0** the rival emitted nothing, G=0 *we* emitted nothing — neither has a finite ratio, both are counted.

Appendix: the engine, isolated from the sampler

			implications/s		implications/sample	
family	targets	n	GPAP	CMSGen	GPAP	CMSGen
ITC'99 + ISCAS-89 seq, T=25	**rare**	13	1.14×10^9	5.20×10^6	30,538	5,422
ITC'99 + ISCAS-89 seq, T=50	**rare**	10	1.07×10^9	5.29×10^6	166,856	11,553
ITC'99 + ISCAS-89 seq, T=25	random	13	1.10×10^9	4.47×10^6	7,254	5,405
ITC'99 + ISCAS-89 seq, T=50	random	9	1.25×10^9	5.38×10^6	14,062	8,867
ISCAS-85 combinational	**rare**	9	1.05×10^9	1.67×10^6	517	507
ISCAS-85 combinational	random	9	1.05×10^9	1.66×10^6	530	535
HWMCC'17, T=25	**rare**	2	1.05×10^9	4.58×10^6	12,615	10,737
HWMCC'17, T=50	**rare**	1	0.87×10^9	1.87×10^6	2,953	2,677
HWMCC'17, T=25	random	2	1.10×10^9	3.43×10^6	3,522	2,910
HWMCC'17, T=50	random	1	0.84×10^9	1.89×10^6	2,253	2,643
EPFL combinational	**rare**	12	1.08×10^9	2.14×10^6	12,139	226
EPFL combinational	random	13	0.89×10^9	4.53×10^6	15,138	630
Sketch / `sk_` projected CNFs	—	27	1.00×10^9	4.23×10^6	851	355

GPAP's rate is from the packed-word kernel; CMSGen's from *its own* counters (`-statsevery`). GPAP holds $\sim10^9$ implications/s on every family — spread only 1.49× — while the sample-rate ratio spans ∼2 orders of magnitude. The two are linked exactly: **samples/s ratio = (impl/s ratio) / (impl/sample ratio)**, so the variation is **rejection cost**, not the propagator. We discard a conflicted trajectory; CMSGen backtracks and keeps the partial work.

Appendix: uniformity against an exact oracle

sampler	circuit		random k-CNF	
	median TV	skewed	median TV	skewed
`ctl:uniform`	0.0395	0/36	0.0401	0/36
`ctl:tv0.05`	0.0577	36/36	0.0580	36/36
`gpap:b1`	0.1109	18/36	0.4368	36/36
`gpap:b8`	0.0401	10/36	0.4033	36/36
`gpap:soft`	0.6255	24/36	0.7605	36/36
`gpap:b32`	0.0399	0/36	0.0401	0/36
`gpap:b64`	0.0399	0/36	0.0401	0/36
`cmsgen`	0.0945	25/36	0.2576	36/36

72 generated instances whose ENTIRE solution set is enumerated by brute force, so the target is known exactly; χ^2 over 3 seeds, Fisher-combined, $\alpha = 0.001$. 0 unsound emissions. **The controls calibrate the instrument**: `ctl:uniform` is exactly uniform and must NOT be flagged; `ctl:tv0.05` is skewed by a known amount and must be. `soft` is why it is opt-in — and $B \geq |\text{support}|$ (b32/b64) is exactly uniform, because discarding conflicts IS rejection sampling.